

Why go to all the trouble of introducing a whole new method that turns out to be equivalent to a method you already knew? The reason is that multivalued methods allow an easy solution to the two difficulties we mentioned above in actually implementing multistep methods.

Consider first the question of stepsize adjustment. To change stepsize from h to h' at some point x_n , simply multiply the components of $\mathbf{y}_n$ in (17.6.5) by the appropriate powers of h'/h , and you are ready to continue to $x_n + h'$.

Multivalued methods also allow a relatively easy change in the *order* of the method: Simply change $\mathbf{r}$. The usual strategy for this is first to determine the new stepsize with the current order from the error estimate. Then check what stepsize would be predicted using an order one greater and one smaller than the current order. Choose the order that allows you to take the biggest next step. Being able to change order also allows an easy solution to the starting problem: Simply start with a first-order method and let the order automatically increase to the appropriate level.

For moderate accuracy requirements, the most efficient choice is almost always a Runge-Kutta routine like `StepperDopr853`. For high accuracy, `StepperBS` is both robust and efficient. For very smooth functions, a variable-order PC method can invoke very high orders. If the right-hand side of the equation is relatively complicated, so that the expense of evaluating it outweighs the bookkeeping expense, then the best PC packages can outperform Bulirsch-Stoer on such problems. As you can imagine, however, such a variable-stepsize, variable-order method is not trivial to program. If you suspect that your problem is suitable for this treatment, we recommend the use of a packaged PC routine. For further details consult [1-3].

Our prediction is that, as extrapolation methods like Bulirsch-Stoer continue to gain sophistication, they will eventually beat out PC methods in *all* applications. We are willing, however, to be corrected.

CITED REFERENCES AND FURTHER READING:

- Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.[1]
Shampine, L.F., and Gordon, M.K. 1975, *Computer Solution of Ordinary Differential Equations. The Initial Value Problem*. (San Francisco: W.H. Freeman).[2]
Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer).[3]
Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 5.
Kahaner, D., Moler, C., and Nash, S. 1989, *Numerical Methods and Software* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 8.
Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 7.

17.7 Stochastic Simulation of Chemical Reaction Networks

We are so used to thinking of chemical (or nuclear) reaction networks as implying sets of continuous differential equations, that it takes an effort to remember

their underlying discrete, atomic, nature. To give an example, we have all learned to translate a set of reactions like



into a set of differential equations (*rate equations*) governing the concentrations of each species,

$$\begin{aligned} \frac{d[A]}{dt} &= -k_0[A][X] \equiv -a_0 \\ \frac{d[X]}{dt} &= k_0[A][X] - k_1[X][Y] \equiv a_0 - a_1 \\ \frac{d[Y]}{dt} &= k_1[X][Y] - k_2[Y] \equiv a_1 - a_2 \\ \frac{d[B]}{dt} &= k_2[Y] \equiv a_2 \end{aligned} \quad (17.7.2)$$

where a_0 , a_1 , and a_2 are respectively the rates of the three reactions in equation (17.7.1).

Increasingly in biological applications, however, one is faced with situations where the actual numbers of reacting molecules is so small that discreteness effects and fluctuations become important. In such cases, one needs to replace continuum concentrations like $[X]$ and $[Y]$ with actual numbers of molecular species. The equations (17.7.2) now become meaningless. What we need to do is directly simulate the discrete reactions in (17.7.1), assigning a sequence of stochastically generated times, and corresponding discrete changes in species numbers, to the occurrences of each reaction. This task is known as *stochastic simulation*, from the original work of Gillespie [1]. Stochastic simulation is a remarkably simple, and elegant, technique. Like many powerful tools, it can be both used and misused, as we will discuss.

Before we get to the details, it is useful to formalize some aspects of the structure of equations (17.7.1) and (17.7.2). In general, we have M reactions occurring among N species. Each reaction $j = 0, \dots, M - 1$, has an instantaneous rate, denoted a_j . In the discrete case, $1/a_j$ is the mean time until the next occurrence of reaction j , if no other reaction happens first. An important point is that each rate a_j depends only on the numbers of those species on the left-hand side of reaction j , its *reactants*. Define a *reactant matrix* λ_{ij} by

$$\lambda_{ij} = \begin{cases} 1 & \text{if species } S_i \text{ is an input to reaction } j \\ 0 & \text{otherwise} \end{cases} \quad (17.7.3)$$

On the output side (*products*), each set of reactions j is characterized by a *state change matrix* v_{ij} whose i, j component is the net change in the number of species S_i due to one occurrence of reaction j . (The j th column of this matrix is often called the *state change vector* for reaction j .) In terms of these quantities, the conventional

rate equations, like (17.7.2), can be written in general as

$$\frac{d[S_i]}{dt} = \sum_{j=0}^{M-1} v_{ij} a_j(\{[S_k]\}), \quad k \text{ s.t. } \lambda_{kj} \neq 0, \quad i = 0, \dots, N-1 \quad (17.7.4)$$

But back to the discrete case: At an instant in time, if we know all the S_i 's, we can compute all the rates a_j . Since rates are additive, the total rate at which *something* will happen is

$$a_{\text{tot}} = \sum_{j=0}^{M-1} a_j \quad (17.7.5)$$

Moreover, because the system is assumed to be “memoryless” (except for the S_j 's) and “well-mixed,” the probability distribution of times to this next occurrence of *some* reaction j must be exponentially distributed (like radioactive decay). Furthermore, given that *some* reaction occurs, it is easy to state what is the probability distribution of *which* reaction it is: It will be reaction j with probability a_j/a_{tot} .

This is all there is, conceptually, to stochastic simulation. The rest is just implementation details, including some clever tricks to speed up the calculation. The steps in the so-called *direct method* are

- From all the S_i 's, compute all the a_j 's and a_{tot} .
- Draw a random number U_1 , uniform in $[0, 1]$, and compute the time τ to the next reaction by

$$\tau = \frac{1}{a_{\text{tot}}} \log \left(\frac{1}{U_1} \right) \quad (17.7.6)$$

(This generates an exponential distribution; cf. §7.3.)

- Draw a second uniform random number U_2 in $[0, 1]$ and find the smallest k such that

$$\sum_{j=0}^k a_j > a_{\text{tot}} U_2 \quad (17.7.7)$$

A value k will thus be chosen with probability a_k/a_{tot} .

- Increment the time t by τ .
- Update each S_i by adding the value v_{ik} .
- Go back to the first step.

17.7.1 Speeding Up the Direct Method

We can speed up the direct method, first, by identifying all steps that are (naively) of $O(M)$ or $O(N)$ and finding ways to make them $O(1)$ (or maybe $\log$); and second, by hand-crafting the inner loop of the program to have the fastest possible execution. The second of these tasks is very important and can make or break a stochastic simulation code's performance; but, unfortunately, it is very machine-, compiler-, and problem-dependent, so it is outside our scope here.

As for the first task, we first note that realistic reaction networks of any size almost always have very sparse reactant and state change matrices: Reactions generally involve only one or two reactants and produce at most a few products. Therefore, it is important to use some kind of sparse matrix structure for the matrices that occur.

When v_{ij} is stored sparsely, for example, the updating step is reduced from $O(N)$ to $O(1)$.

Next, we note that most a_j 's are unchanged after each reaction occurs. After a reaction k , for example, the only a_j 's that need to be recomputed are those with reactants (inputs) that were changed by a nonzero entry in the k th column of the v_{jk} matrix. A way to formalize this is by a *dependency graph* or *dependency matrix* G , whose component G_{jk} is nonzero only if reaction k changes a species that is an input to reaction j . With a moment of thought, you will figure out that the matrix G can be obtained by the logical matrix multiplication of λ^T and v , namely

$$G_{ij} = \bigcup_k \lambda_{ki} \cap v_{kj} \quad (17.7.8)$$

where $\cup$ denotes logical-or, $\cap$ denotes logical-and, and the C convention of “true iff nonzero” is assumed. Now, after each reaction j , we only update the a_i 's indicated by the j th column of G_{ij} . Of course we must also store G in a sparse format.

Finally, there is the question of how to speed up the choice of which reaction to update, equation (17.7.7), which can be at worst $O(M)$. Here there are two schools of thought. The one that we implement below, following advice in [3], takes advantage of the fact that for many, if not most, actual applications, a small number of reactions ($\ll M$) dominate the reaction rates. If we arrange the order of the a_j 's in equation (17.7.7) with these dominant reactions first, then it can take, on average, only $O(1)$ tests to select the next reaction. In [3], it is suggested to do preliminary runs to find which reactions dominate. We prefer the more transparent alternative, implemented below, of just letting frequent reactions adaptively bubble up in a priority list.

The other school of thought, called the *next reaction method* [2], is discussed separately, below. It cleverly changes $O(M)$ to something like $O(\log M)$, even in the most unfavorable case. However, the number of operations in the inner program loop is significantly larger than for the (optimized, as above) direct method. Which method is fastest is very likely problem- and implementation-dependent.

For the modest test case illustrated, namely the set of three reactions (17.7.1), most of the optimizations illustrated in the following code are unnecessary, and likely counterproductive. However, the intent is to be illustrative of what a code for larger problems would look like.

```
struct Stochsim {
```

stochsim.h

Object for stochastic simulation of a set of chemical reactions.

<code>VecDoub s;</code>	Vector of species numbers.
<code>VecDoub a;</code>	Vector of rates.
<code>MatDoub instate, outstate;</code>	
<code>NRvector<NRsparseCol> outchg, depend;</code>	Sparse matrices v_{ij} and G_{ij}
<code>VecInt pr;</code>	Priority list.
<code>Doub t, asum;</code>	
<code>Ran ran;</code>	
<code>typedef Doub(Stochsim::*rateptr)();</code>	Obscure C++ used to create a vec-
<code>rateptr *dispatch;</code>	tor dispatch of function pointers to the rate functions.

```
// begin user section
```

Replace this section, using as a template the example (17.7.1) shown here, by the particulars of your reaction network. If you have a large number of reactions, you will want to generate the matrices `instate` and `outstate` externally, and pass them as globals (or read them here).

<code>static const Int mm=3;</code>	Set number of reactions.
<code>static const Int nn=4;</code>	Set number of species.

```

Doub k0,k1,k2;
Doub rate0() {return k0*s[0]*s[1];}
Doub rate1() {return k1*s[1]*s[2];}
Doub rate2() {return k2*s[2];}
void describereactions () {
    You provide a function with this name that sets any constants that you have defined and
    sets the instate and outstate matrices to describe your reactions.

    k0 = 0.01;
    k1 = .1;
    k2 = 1.;
    Doub indat[] = {
        1.,0.,0.,
        1.,1.,0.,
        0.,1.,1.,
        0.,0.,0.
    };
    instate = MatDoub(nn,mm,indat);
    Doub outdat[] = {
        -1.,0.,0.,
        1.,-1.,0.,
        0.,1.,-1.,
        0.,0.,1.
    };
    outstate = MatDoub(nn,mm,outdat);
    dispatch[0] = &Stochsim::rate0;
    dispatch[1] = &Stochsim::rate1;
    dispatch[2] = &Stochsim::rate2;
}
// end user section

Stochsim(VecDoub &sinit, Int seed=1)
Constructor. Input initial species numbers and an optional random seed.
: s(sinit), a(mm,0.), outchg(mm), depend(mm), pr(mm), t(0.),
asum(0.), ran(seed), dispatch(new rateptr[mm]) {
    Int i,j,k,d;
    describereactions();
    sparmatfill(outchg,outstate);
    MatDoub dep(mm,mm);
    for (i=0;i<mm;i++) for (j=0;j<mm;j++) {
        d = 0;
        for (k=0;k<nn;k++) d = d || (instate[k][i] && outstate[k][j]);
        dep[i][j] = d;
    }
    sparmatfill(depend,dep);
    for (i=0;i<mm;i++) {
        pr[i] = i;
        a[i] = (this->dispatch[i])();
        asum += a[i];
    }
}
~Stochsim() {delete [] dispatch;}

Doub step() {
    Take a single stochastic step (one reaction) and return the new time.
    Int i,n,m,k=0;
    Doub tau,atarg,sum,anew;
    if (asum == 0.) {t += 2.; return t;}
    tau = -log(ran.doub())/asum;
    atarg = ran.doub()*asum;
    sum = a[pr[0]];
    while (sum < atarg) sum += a[pr[++k]];
    m = pr[k];
    if (k > 0) SWAP(pr[k],pr[k-1]);
    if (k == mm-1) asum = sum;

```

Declare any rate constants needed.
Your rate functions go here.

The reactant matrix λ_{ij} .

The state change matrix v_{ij} .

You must also point the dispatch table entries to the correct rate functions.

Logical matrix multiply calculates the dependency matrix.

Calculate all initial rates.

Rare: All reactions have stopped exactly, so double the time until the user notices!

Equation (17.7.7).

Move reaction up on the priority list.
Free update of asum fixes accumulated roundoff.

```

    n = outchg[m].nvals;
    for (i=0;i<n;i++) {
        k = outchg[m].row_ind[i];
        s[k] += outchg[m].val[i];
    }
    n = depend[m].nvals;
    for (i=0;i<n;i++) {
        k = depend[m].row_ind[i];
        anew = (this->*dispatch[k])();
        asum += (anew - a[k]);
        a[k] = anew;
    }
    if (t*asum < 0.1)
        for (asum=0.,i=0;i<mm;i++) asum += a[i];
    return (t += tau);
}
};

```

Apply state change vector.

Recalculate rates required by dependency matrix.

Rare: Rates heading toward zero.
Better recalculate asum.

Note that Stochsim uses some arcane C++ syntax (“array of pointers to member functions”) in connection with the identifier `dispatch`. The underlying idea is simple, and important: We want to jump directly to the appropriate user-supplied rate function, as directed by an integer index. There are various ways of coding this, but what you *don’t* want to have is a long chain of `if` tests that would be $O(M)$ instead of $O(1)$. (Perhaps we should believe that C’s `switch` statement is always properly implemented by compilers as a fast table dispatch, but we don’t.)

The utility routine that constructs a sparse matrix out of a full one is this (cf. §2.7):

```

void sparmatfill(NRvector<NRsparseCol> &sparmat, MatDoub &fullmat) {
    Utility that fills a sparse matrix from a full one. See §2.7.
    int n,m,nz,nn=fullmat.nrows(),mm=fullmat.ncols();
    if (sparmat.size() != mm) throw("bad sizes");
    for (m=0;m<mm;m++) {
        for (nz=n=0;n<nn;n++) if (fullmat[n][m]) nz++;
        sparmat[m].resize(nn,nz);
        for (nz=n=0;n<nn;n++) if (fullmat[n][m]) {
            sparmat[m].row_ind[nz] = n;
            sparmat[m].val[nz++] = fullmat[n][m];
        }
    }
}

```

stochsim.h

As a cultural note, the system (17.7.1) is not just any old chemical reaction network, but is actually a form of the *Lotka-Volterra* equation, discovered independently by Alfred J. Lotka and Vito Volterra in 1925–1926. In fact, it’s not originally a chemical reaction network at all, but a set of relationships intended to model predator-prey relationships. The first equation says, roughly, that rabbits (X) eat grass (A) to produce more rabbits. The second says that foxes (Y) eat rabbits (X) to produce more foxes. The third says that foxes don’t live forever. (For some reason, rabbits do live forever in this model, unless they are eaten by foxes.)

Figure 17.7.1 shows an example of the system’s evolution, starting with initial conditions $A = 150$, $X = Y = 10$, and $B = 0$. One sees two cycles of prey population growth, with predator growth following, and then a collapse of both populations. After the second cycle, by a fluctuation, the predator population goes to zero, from which it cannot recover. At the end of the evolution shown, the prey population is starting to recover; but it is not a happy ending, because, by now, the food supply is running out. The world of stochastic simulation is a harsh one. Stochastic ef-

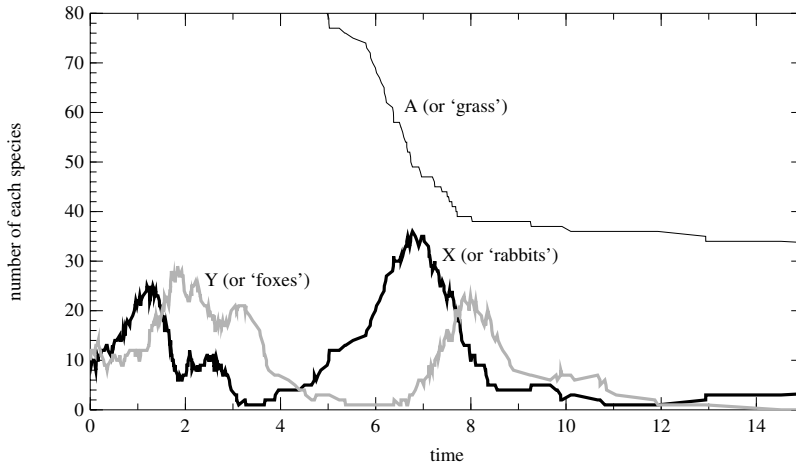


Figure 17.7.1. Evolution of the reaction network (17.7.1). This network evolves by the Lotka-Volterra equations, originally developed as a model for predator-prey interactions. Stochastic effects are important; with different random seeds, different time histories occur.

fects are genuinely dominant in this example: Exactly the same equations and initial conditions, but with a different random seed, give entirely different evolutions.

17.7.2 Next Reaction Method

The *next reaction method* [2] starts by computing not a single reaction time, equation (17.7.6), but rather a separate next reaction time for each reaction j ,

$$\tau_j = \frac{1}{a_j} \log \left(\frac{1}{U_j} \right) \quad (17.7.9)$$

where the U_j 's are independent uniform random deviates in $[0, 1]$. These times are all stored in a heap (see §8.3), so that the smallest value can be easily accessed at the top of the heap (call it k). The following steps are now repeatedly executed:

- Do reaction k and update the affected S_i 's (using the matrix v_{ik}). Increment time t by τ_k .
- Compute a next time for reaction k (using equation 17.7.9 and adding t) and store it on the heap.
- For every affected reaction j (as determined by a nonzero entry G_{jk}), correct its stored next time by the formula

$$\tau_j \leftarrow \frac{a_{j,\text{old}}}{a_{j,\text{new}}} (\tau_j - t) + t \quad (17.7.10)$$

This is called *time reuse*. In effect, it reuses the random deviate U_j that originally generated τ_j , but it corrects the resulting time prediction for an intermediate step function change in a_j . Sounds dodgy, we know, but it is probabilistically sound.

- Get the heap back into order by bubbling elements up or down as required. This is where the complexity of the inner loop gets increased, to as much as $O(\log M)$.

Unquestionably, one can construct reaction networks for which the next reaction method is considerably faster than the optimized direct method. However, networks dominated by a small number of fast reactions are so common in practice that this performance advantage should not be assumed a priori [3].

17.7.3 Practical Advice

Don't ever use a stochastic simulation method — of any flavor — unless your problem is genuinely stochastic. Instead, use the deterministic rate equations (17.7.4) with a good stiff equation solver like `StepperSie` in §17.5. Such solvers are not limited by the rate of the fastest reaction, and will frequently be orders of magnitude faster than any stochastic method. (We are reliably informed that an unconscionable number of CPU hours are wasted by misguided researchers who think that the stochastic simulation method is an all-purpose tool for reaction networks.)

Just to show you how easy this is, here is how you would do the Lotka-Volterra problem (17.7.2) by integrating the equations directly. First encode the right-hand side $\mathbf{f}$ and the Jacobian of the right-hand side in a structure. (The (i, j) element of the Jacobian is $\partial f_i / \partial y_j$.)

```
struct rhs {
    Doub k0,k1,k2;
    rhs(Doub kk0, Doub kk1, Doub kk2) : k0(kk0),k1(kk1),k2(kk2) {}
    void operator()(const Doub x, VecDoub_I &y, VecDoub_O &dydx) {
        dydx[0]= -k0*y[0]*y[1];
        dydx[1]= k0*y[0]*y[1]-k1*y[1]*y[2];
        dydx[2]= k1*y[1]*y[2]-k2*y[2];
        dydx[3]= k2*y[2];
    }
    void jacobian(const Doub x, VecDoub_I &y, VecDoub_O &dxdx,
                  MatDoub_O &dxdy) {
        Int n=y.size();
        for (Int i=0;i<n;i++) dxdx[i]=0.0;
        dxdy[0][0] = -k0*y[1];
        dxdy[0][1] = -k0*y[0];
        dxdy[0][2] = 0.0;
        dxdy[0][3] = 0.0;
        dxdy[1][0] = k0*y[1];
        dxdy[1][1] = k0*y[0]-k1*y[2];
        dxdy[1][2] = -k1*y[1];
        dxdy[1][3] = 0.0;
        dxdy[2][0] = 0.0;
        dxdy[2][1] = k1*y[2];
        dxdy[2][2] = k1*y[1]-k2;
        dxdy[2][3] = 0.0;
        dxdy[3][0] = 0.0;
        dxdy[3][1] = 0.0;
        dxdy[3][2] = k2;
        dxdy[3][3] = 0.0;
    }
};
```

Next set the parameters for `Odeint`, for example

```
const Int n=4;
Doub rtol=1.0e-7,atol=1.0e-4*rtol,h1=1.0e-6,hmin=0.0,x1=0.0,x2=15.0;
VecDoub ystart(n);
ystart[0]=150.0;
ystart[1]=10.0;
```



```

ystart[2]=10.0;
ystart[3]=0.0;
Output out(100);           Output at 100 uniform points
rhs d(0.01,0.1,1.0);       Declare d as a rhs object.
Odeint<StepperSIE<rhs> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
ode.integrate();

```

Note how the values of k_0 , k_1 , and k_2 are passed as arguments in the constructor call that declares `d`. These particular values don't make the system of equations particularly stiff, so you could use a standard integrator. However, this is not true in general for real-world examples.

The output, which is equally spaced, can be printed by statements like

```

for (Int i=0;i<out.count;i++)
    cout << out.xsave[i] << " " << out.ysave[0][i] << " " <<
        out.ysave[1][i] << " " << out.ysave[2][i] << endl;

```

If your network's fastest reactions are not stochastic, but there are some slower reactions where stochastic effects are important, then look into so-called hybrid methods (e.g., [4]).

CITED REFERENCES AND FURTHER READING:

- Gillespie, D.T. 1976, "A General Method for Numerically Simulating the Stochastic Time Evolution of Coupled Chemical Reactions," *Journal of Computational Physics*, vol. 11, pp. 403–434.[1]
- Gibson, M.A., and Bruck, J. 2000, "Efficient Exact Stochastic Simulation of Chemical Systems with Many Species and Many Channels," *Journal of Physical Chemistry A*, vol. 104, pp. 1876–1889.[2]
- Cao, Y., Li, H., and Petzold, L. 2004, "Efficient Formulation of the Stochastic Simulation Algorithm for Chemically Reacting Systems," *Journal of Chemical Physics*, vol. 121, pp. 4059–4067.[3]
- Salis, H., and Kaznessis, Y. 2005, "Accurate Hybrid Stochastic Simulation of a System of Coupled Chemical or Biochemical Reactions," *Journal of Chemical Physics*, vol. 122, art. 054103.[4]